

Какво да запомним? Подготовка за изпитване

Въпроси:

ASP = Active Server Page – интернет страница с HTML и скриптове, за уеб приложения.

ASP .Net Core е крос-платформа (работи на Windows и Linux), с висока производителност, отворен код за създаване на модерни, облачно-базирани, свързани с интернет приложения.

SDK = Software Development Kit – колекция от софтуерни инструменти в един инсталиран пакет.

Runtime = Run only – само стартира проект, но не може да го създаде.

Core – ядро - съдържа всичко в себе си (може да го проектира и стартира с Runtime).

* Empty – празен, ние трябва да пишем всичко

* MVC – Model-View-Controller – генерира най-много авто-код. Съдържа всичко наготово генерирано – правила сме.

* Razor = ножица, резачка - изрезка от сходен код, шаблон по образец.

HTML = HyperText Markup Language – език за маркиране на текст с хиперлинкове за уеб сайтове.

http = HyperText Transfer Protocol – протокол за трансфер на HTML страници в интернет.

https = http + s, където S = Secured, защитен http, който е важен за Login с пароли, за да не се покажат в адрес-бара (навигационната лента) на брауъра.

.NetCore app – започва като конзолно приложение (терминала) и по-късно се развива като web.

InProcess има много по-голяма пропускателна способност от OutOfProcess.

Имаме 2 локални сървъра: вътрешен (Kestrel) + външен (iis express). Когато разработваме проект (т.е. development) искаме да имаме по-голяма видимост, затова използваме и двата сървъра (вътрешният междинен ни помага за контрол). Когато, обаче, публикуваме официално сайта и всичко е готово и без грешки (InProduction) искаме бързина за потребителите, затова средният слой (сървър) става излишен и използваме само външния.

OutOfProcess = 2 сървъра (вътрешен Kestrel + външен IIS Express), затова е по-бавен, Development

InProcess = 1 сървър (външен IISExpress) – много по-бърз.

InProduction използваме iis

IIS = Information Internet Serrver

Възможните стадии при разработка на проект или още сред Environment са 3:

Development – по време на разработка

Staging – финални изпитания

Production – официално публикуване за ползване от потребителите в интернет.

Направили сме празен Empty проект, без .https отметка, нарекли сме го слято без интервал EmployeeManagement, отваряме папката на проекта от Visual Code, от терминала стартираме с **dotnet run**. Ако не тръгне веднага, търсим директорията до Program.cs файла със **cd .** и табулации с Tab докато изпише EmployeeManagement и опитваме отново. Ако не стане, влизаме още една директория навътре.

CMD = Command Prompt – конзолата / терминала, в който се въвеждат команди (черния прозорец)
CLI = Command Line Interpreter – CMD на Visual Studio се нарича така.

cd = change directory

/d – указва адрес на директорията (пътя)

. - влиза 1 директория навътре в папката. Може да се използва клавиш Tab за автоматично дописване

.. - излиза 1 директория нагоре (качва се) в папката.

! Всеки път, когато променим код, даваме

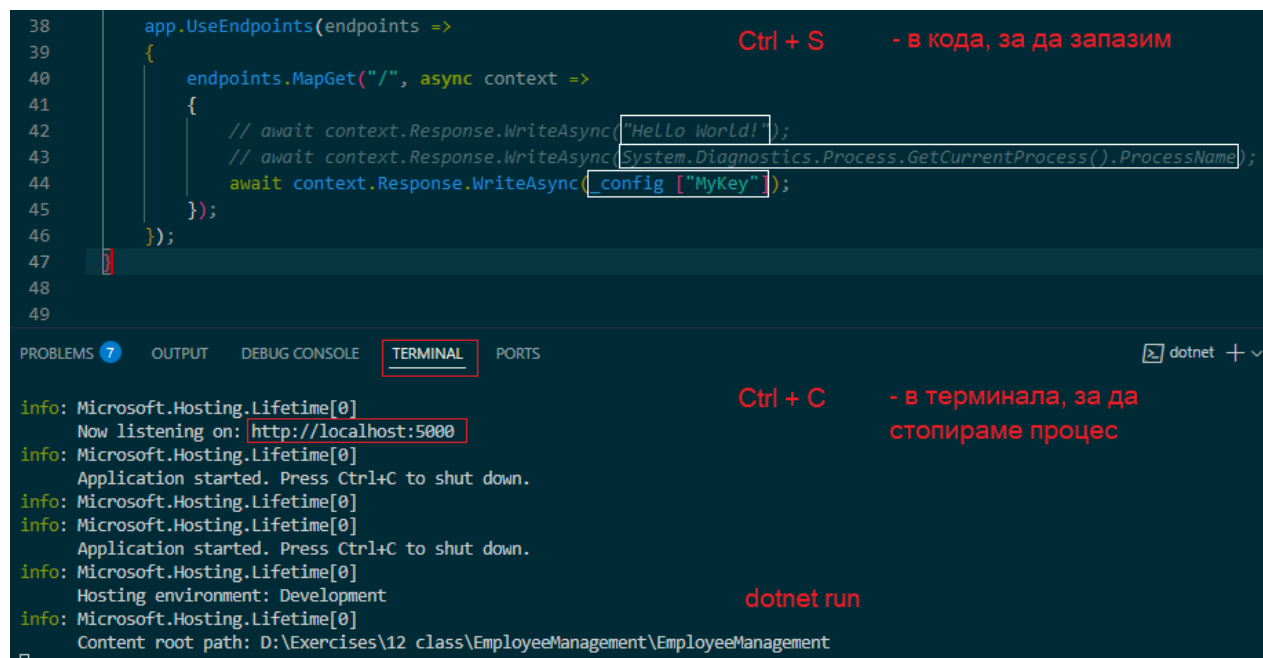
Ctrl + S = Save

Ctrl + C = Close

dotnet run

Ctrl + R = Refresh

Много е важно, че Ctrl + S се дава, когато курсорът е в кода, а Ctrl + C, за да спрем терминала, когато курсорът е в самия терминал. Няма да стане, ако натискам едното, а сме селектирали другаде.



The screenshot shows the Visual Studio IDE. The top part displays C# code for a web application endpoint. The bottom part shows the 'TERMINAL' tab with the output of the application running.

```
38 app.UseEndpoints(endpoints =>
39 {
40     endpoints.MapGet("/", async context =>
41     {
42         // await context.Response.WriteAsync("Hello World!");
43         // await context.Response.WriteAsync(System.Diagnostics.Process.GetCurrentProcess().ProcessName);
44         await context.Response.WriteAsync(config["MyKey"]);
45     });
46 });
47
48
49
```

Annotations on the code:

- Ctrl + S** - в кода, за да запазим (next to line 38)

The terminal output shows the application starting and listening on `http://localhost:5000`. It also shows the environment variables.

```
info: Microsoft.Hosting.Lifetime[0]
Now listening on: http://localhost:5000
info: Microsoft.Hosting.Lifetime[0]
Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
Content root path: D:\Exercises\12 class\EmployeeManagement\EmployeeManagement
```

Annotations on the terminal:

- Ctrl + C** - в терминала, за да стопираме процес (next to the first info line)
- dotnet run** (next to the environment info line)

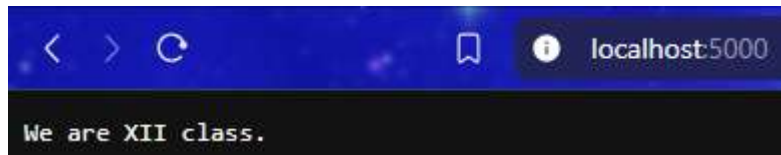
Задачи

1 зад: Изведете съобщение на екрана „We are XII class.“

Решение: В Startup.cs махаме коментара и вместо “Hello world!” изписваме

`await context.Response.WriteAsync("We are XII class.");`

```
app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        await context.Response.WriteAsync("We are XII class.");
        // await context.Response.WriteAsync(System.Diagnostics.Process.GetCurrentProcess().ProcessName);
        // await context.Response.WriteAsync(_config ["MyKey"]);
    });
});
```



2 зад: Изведете процеса dotnet в брауъра.

Решение: Закоментираме ненужните редове и оставяме системната диагностика между скобите за изписване на екрана:

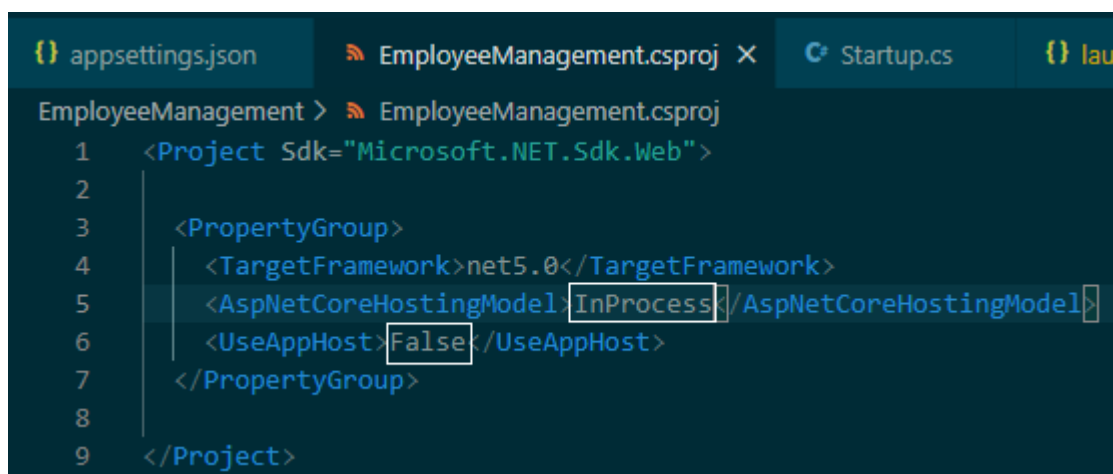
```
endpoints.MapGet("/", async context =>
{
    //await context.Response.WriteAsync("We are XII class.");
    await context.Response.WriteAsync(System.Diagnostics.Process.GetCurrentProcess().ProcessName);
    // await context.Response.WriteAsync(_config ["MyKey"]);
});
```

`await context.Response.WriteAsync(System.Diagnostics.Process.GetCurrentProcess().ProcessName);`

Реално, вместо "Hello world!", в скобите сме записали

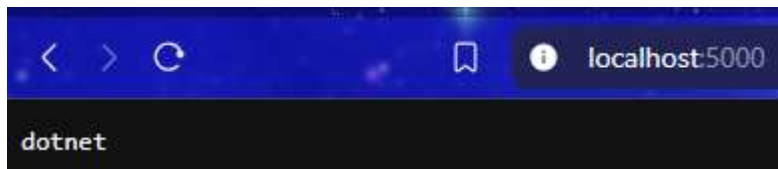
`System.Diagnostics.Process.GetCurrentProcess().ProcessName` с което извикваме системния процес, но не се слагат кавички, иначе ще стане обикновен текст/стринг и няма да носи информация.

Отиваме в EmployeeManagement.csproj и настройваме InProcess, False



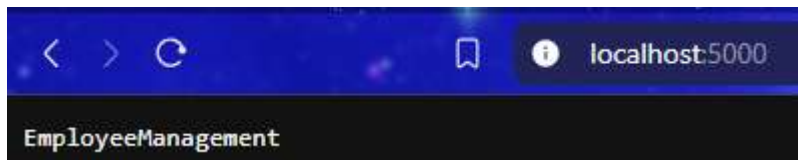
`<AspNetCoreHostingModel>InProcess</AspNetCoreHostingModel>`

`<UseAppHost>False</UseAppHost>`



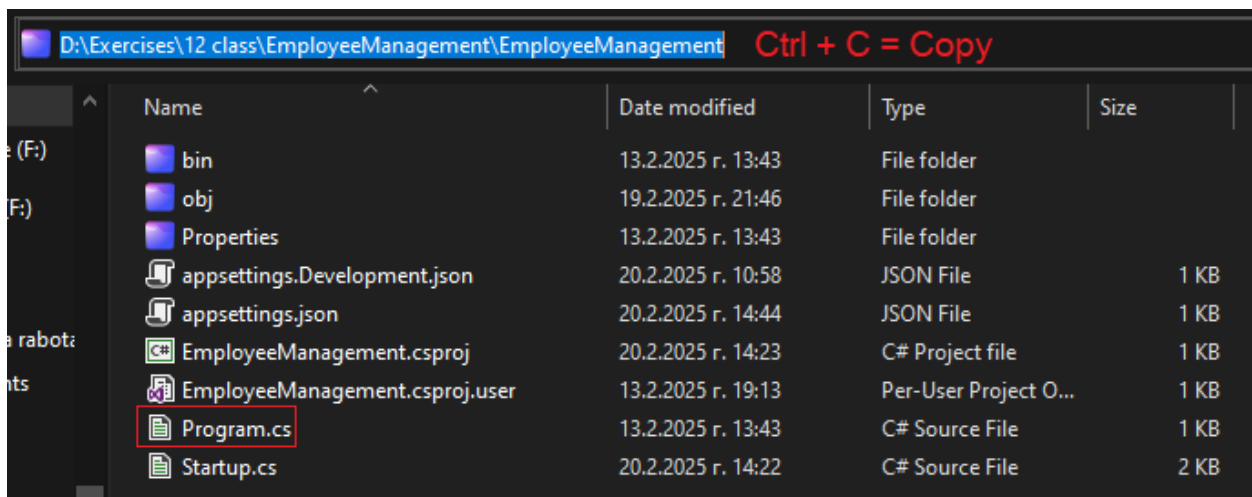
3 зад: Изведете в брауъра името на процеса, който се извиква от вътрешния (Kestrel) сървър.

```
<AspNetCoreHostingModel>OutOfProcess</AspNetCoreHostingModel>  
<UseAppHost>True</UseAppHost>
```



4 зад: Стартирайте проекта през CMD / CLI.

Решение: Първо копираме адреса в навигационната лента на Windows Explorer в нашата папка до Program.cs файла.



В търсачката пишем **CMD** и стартираме.

Чрез **cd /d** отиваме в указаната директория, която сме копирали и я поставяме с Ctrl + V.

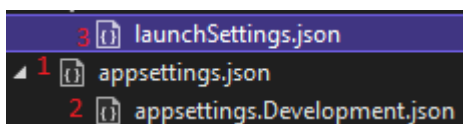
После стартираме проекта с dotnet run.

```
Select Command Prompt - dotnet run

C:\Users\Миконета>cd /d D:\Exercises\12 class\EmployeeManagement\EmployeeManagement cd /d Ctrl + V = Paste
D:\Exercises\12 class\EmployeeManagement\EmployeeManagement>dotnet run
Using launch settings from D:\Exercises\12 class\EmployeeManagement\EmployeeManagement\Properties\launchSettings.json...
Building...
C:\Program Files\dotnet\sdk\9.0.200\Sdks\Microsoft.NET.Sdk\targets\Microsoft.NET.EolTargetFrameworks.targets(32,5): warn
ing NETSDK1138: The target framework 'net5.0' is out of support and will not receive security updates in the future. Ple
ase refer to https://aka.ms/dotnet-core-support for more information about the support policy.
C:\Program Files\dotnet\sdk\9.0.200\Sdks\Microsoft.NET.Sdk\targets\Microsoft.NET.EolTargetFrameworks.targets(32,5): warn
ing NETSDK1138: The target framework 'net5.0' is out of support and will not receive security updates in the future. Ple
ase refer to https://aka.ms/dotnet-core-support for more information about the support policy.
C:\Program Files\dotnet\sdk\9.0.200\Sdks\Microsoft.NET.Sdk\targets\Microsoft.NET.EolTargetFrameworks.targets(32,5): warn
ing NETSDK1138: The target framework 'net5.0' is out of support and will not receive security updates in the future. Ple
ase refer to https://aka.ms/dotnet-core-support for more information about the support policy.
info: Microsoft.Hosting.Lifetime[0]
      Now listening on: http://localhost:5000
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
```

Важно е да няма стартирани 2 терминала едновременно!!! Ако някъде другаде Visual Studio / Code има работещ терминал, трябва да го спрем с Ctrl + C или затворим, защото иначе ще хвърля грешки. Не може да работи повече от 1 терминал! Терминалът е с най-висок приоритет и засенчва (доминира върху) останалите процеси и настройки.

5 зад: Демонстрирайте приоритетите в настройващите .json файлове.



- 1) Appsettings.json (min)
- 2) Appsettings.{Environment}.json където на мястото на {Environment} могат да стоят Development (в нашия случай), Staging или Production. При нас, фалът е Appsettings.Development.json
- 3) User secrets
- 4) Environment variables – launchsettings.json
- 5) Command-line arguments CMD/CLI (max)

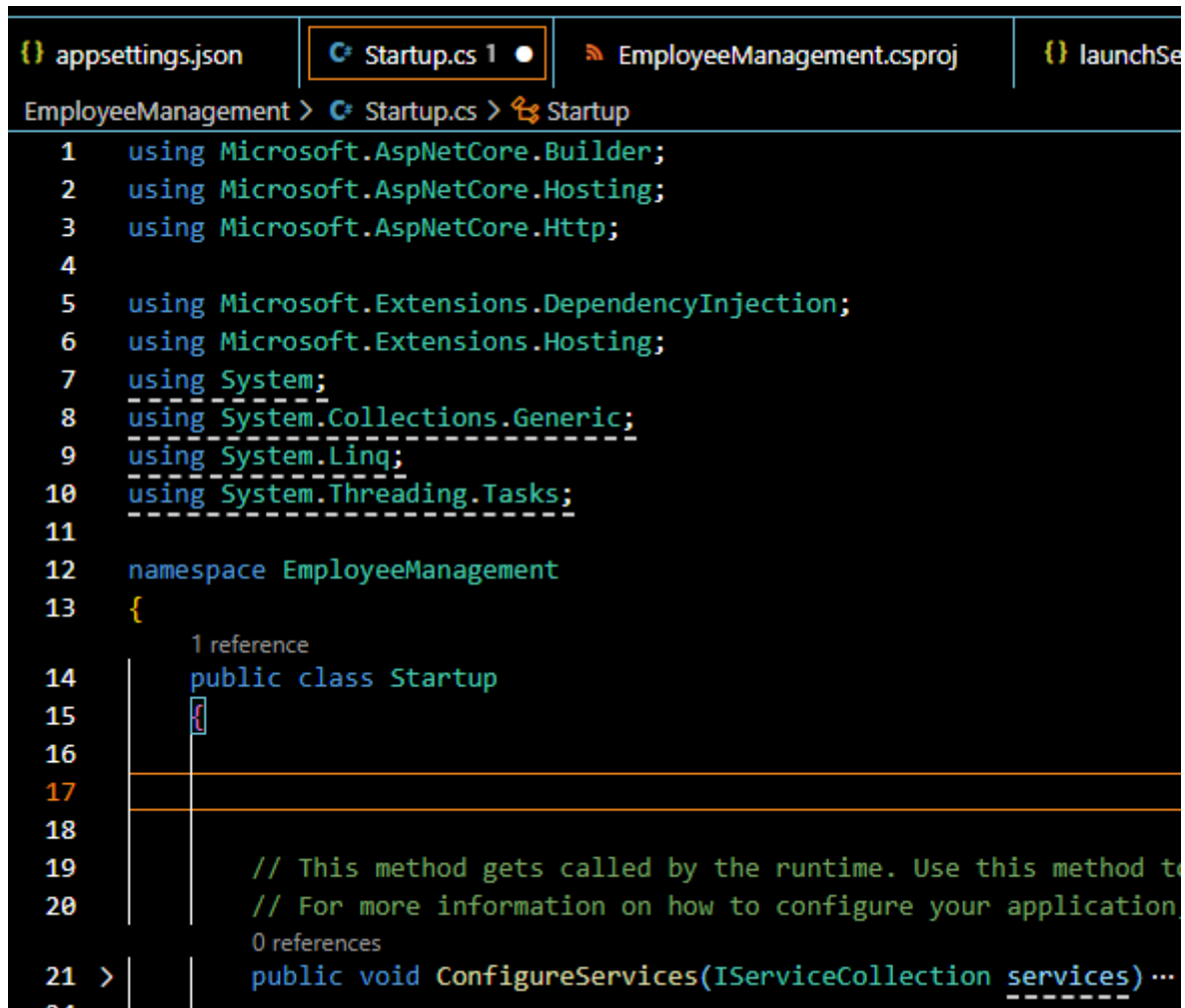
В нашия Startup.cs решаваме да изпишем на екрана конфигурационна променлива `_config`, която сме нарекли `MyKey`:

```
await context.Response.WriteAsync(_config ["MyKey"]);
```

```
endpoints.MapGet("/", async context =>
{
    //await context.Response.WriteAsync("Hello world!");
    // await context.Response.WriteAsync(System.Diagnostics.Process.GetCurrentProcess().ProcessName);
    await context.Response.WriteAsync(_config ["MyKey"]);
});
```

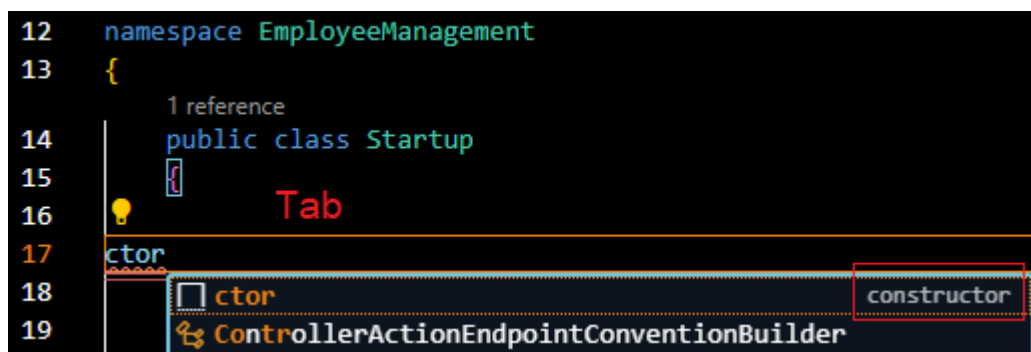
Реално, вместо "Hello world!", в скобите сме записали `_config ["MyKey"]` като команда да се изпечата на екрана.

Качваме се най-нагоре в class Startup и си освобождаваме място.



```
1 using Microsoft.AspNetCore.Builder;
2 using Microsoft.AspNetCore.Hosting;
3 using Microsoft.AspNetCore.Http;
4
5 using Microsoft.Extensions.DependencyInjection;
6 using Microsoft.Extensions.Hosting;
7 using System;
8 using System.Collections.Generic;
9 using System.Linq;
10 using System.Threading.Tasks;
11
12 namespace EmployeeManagement
13 {
14     public class Startup
15     {
16
17
18
19         // This method gets called by the runtime. Use this method to
20         // For more information on how to configure your application
21         public void ConfigureServices(IServiceCollection services) ...
```

С команда **ctor + Tab** създаваме празен конструктор, който носи името на класа, в който се намира – в случая Startup



```
12 namespace EmployeeManagement
13 {
14     public class Startup
15     {
16         public Startup()
17     }
18
19     public void ConfigureServices(IServiceCollection services) ...
```

Автоматично се генерира празен конструктор:

```
public Startup(Parameters)
```

```
{
```

}

```
12 namespace EmployeeManagement
13 {
14     2 references
15     public class Startup
16     {
17         0 references
18         public Startup(Parameters)
19     {
20     }
21
22     // This method gets cal
```

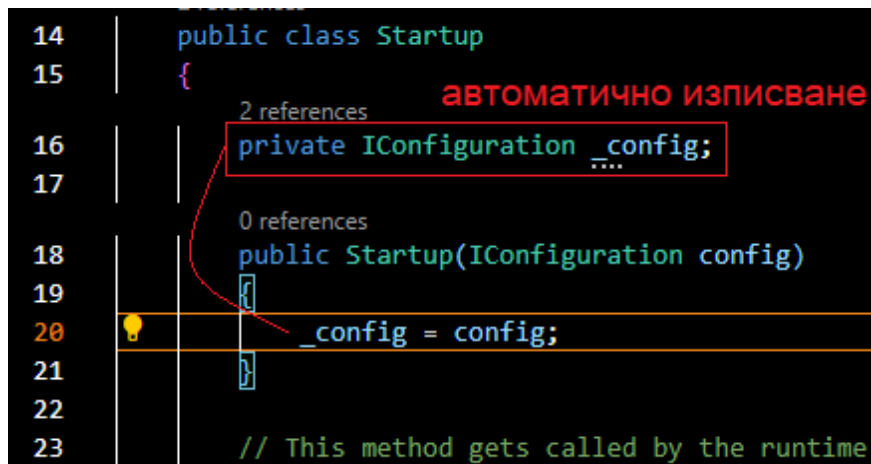
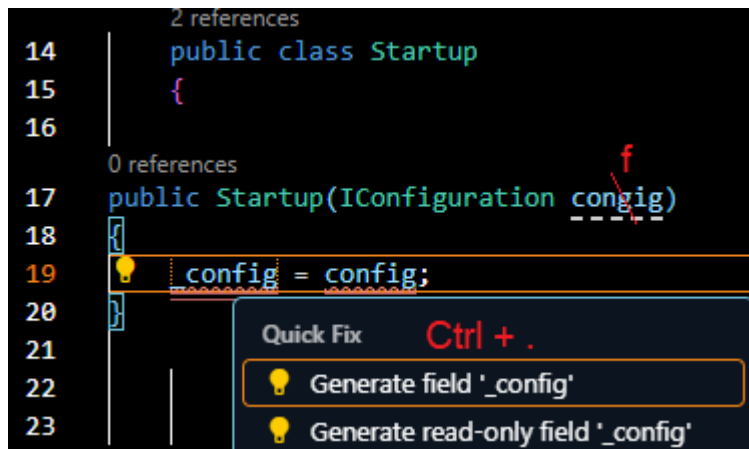
Извикваме в него конфигурационна променлива, като в IConfiguration, началната буква I винаги означава Interface. На секундата, в която изпишем IConfiguration, веднага се появява библиотеката, която позволява използването му горе -> using Microsoft.Extensions.Configuration;

```
3 using Microsoft.AspNetCore.Http;
4 using Microsoft.Extensions.Configuration;
5 using Microsoft.Extensions.DependencyInjection;
6 using Microsoft.Extensions.Hosting;
7 using System;
8 using System.Collections.Generic;
9 using System.Linq;
10 using System.Threading.Tasks;
11
12 namespace EmployeeManagement
13 {
14     2 references
15     public class Startup
16     {
17         0 references
18         public Startup(IConfiguration config)
19     {
20     }
21 }
```

IntelliSense = Intelligent + Sense = (интелигентно изречение) – автоматичното дописване на код. Желателно е да го използваме колкото може повече с потвърждения с Tab или Enter, т.к. така вероятността от сгрешена буква е много по-малък и кодът се пише по-бързо.

Влизаме в конструктора и между двете къдрави скоби { } задаваме използването на `_config = config;` Подчертава двете думи с червена зигзагообразна линия за грешка и лампичка за

насочване. Когато посочим с курсора върху думата `_config` и натиснем клавишна комбинация `Ctrl + .` (`Ctrl` и точка едновременно), което е стандартната команда за автоматичен `Debug`, веднага генерира частно поле за заделяне на памет над конструктора нагоре, необходима за тази променлива.



Разбира се, другият начин е да изпишем всичко ръчно:

```
private IConfiguration _config;
```

```
public Startup(IConfiguration config)
```

```
{
```

```
    _config = config;
```

```
}
```

като не забравяме да декларираме и необходимата библиотека най-високо – трябва да проверим дали я има. За да работи `IConfiguration`, трябва да имаме и

```
using Microsoft.Extensions.Configuration;
```

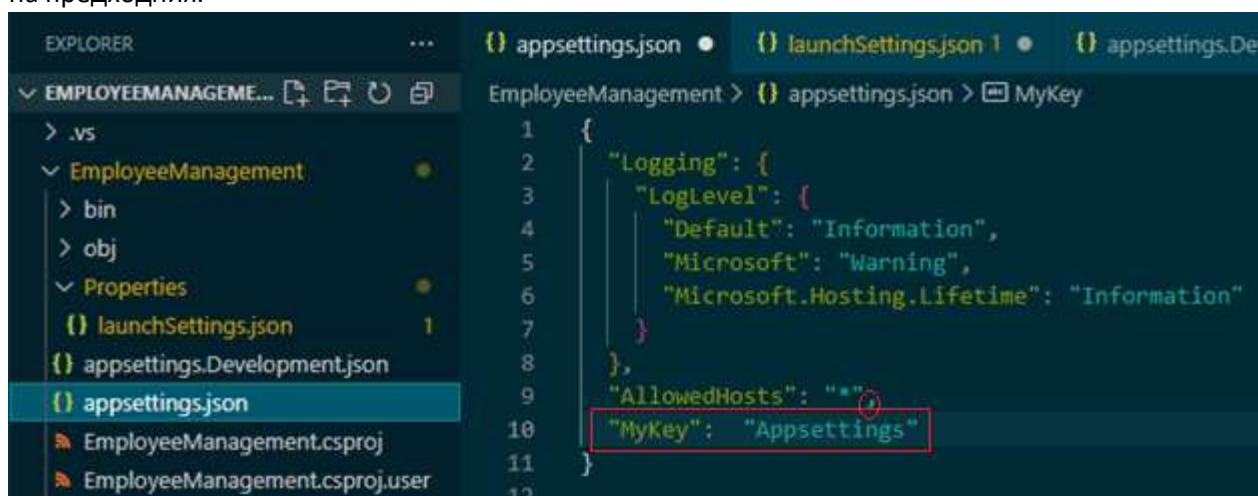
След като всички тези неща в главния `Startup.cs` файл са готови, за да можем да изпишем тази конфигурационна променлива `MyKey`:


```

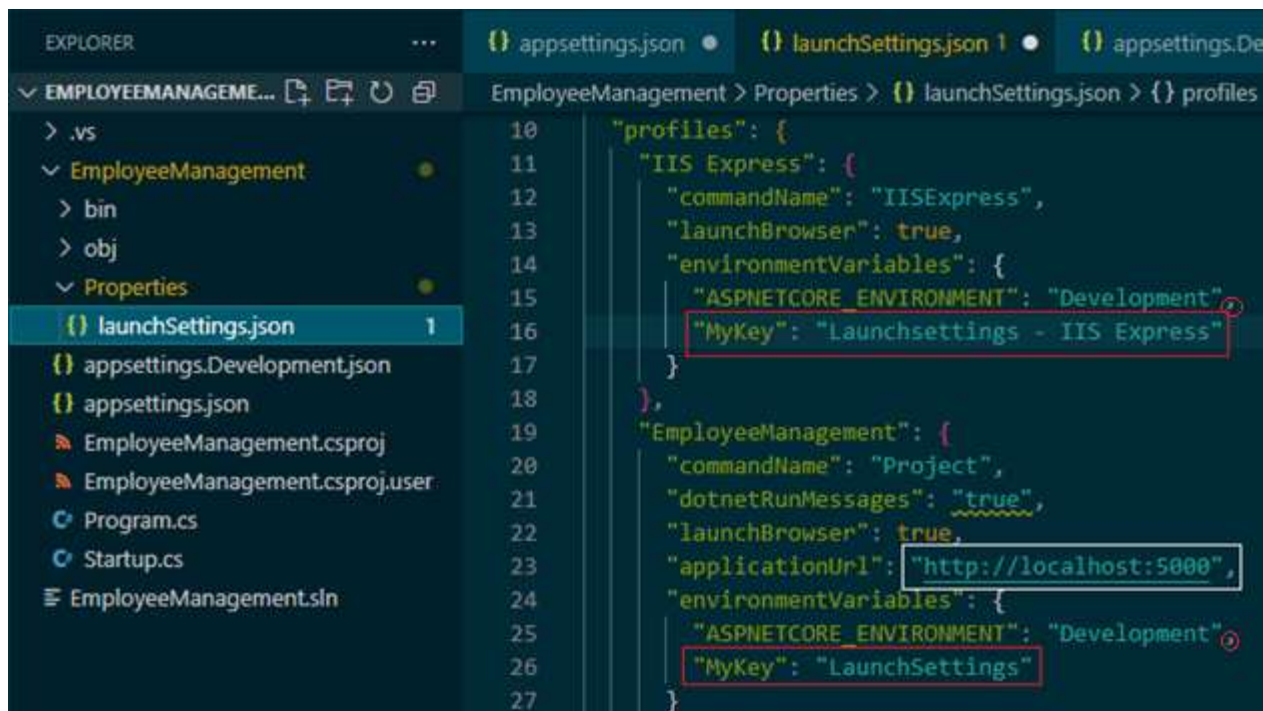
38 app.UseEndpoints(endpoints =>
39 {
40     endpoints.MapGet("/", async context =>
41     {
42         // await context.Response.WriteAsync("Hello World!");
43         //await context.Response.WriteAsync(System.Diagnostics.Process.GetCurrentProcess().ProcessName);
44         await context.Response.WriteAsync(_config["MyKey"]);
45     });
46 };

```

отиваме да я зададем в 3-те .json файла – appsettings.json, launchSettings.json, appsettings.Development.json. Текстът в кавичките е по наш избор – можем да изпишем каквото искаме да се покаже на екрана в браузъра, важното е да може да се ориентираме кой файл е активен (с най-висок приоритет спрямо другите) в дадения момент от настройките. Не забравяме, че по същия начин както в SQL, преди да започнем да пишем на новия ред, слагаме запетайка за край на предходния.



"MyKey": "Hello from Appsettings.json!"

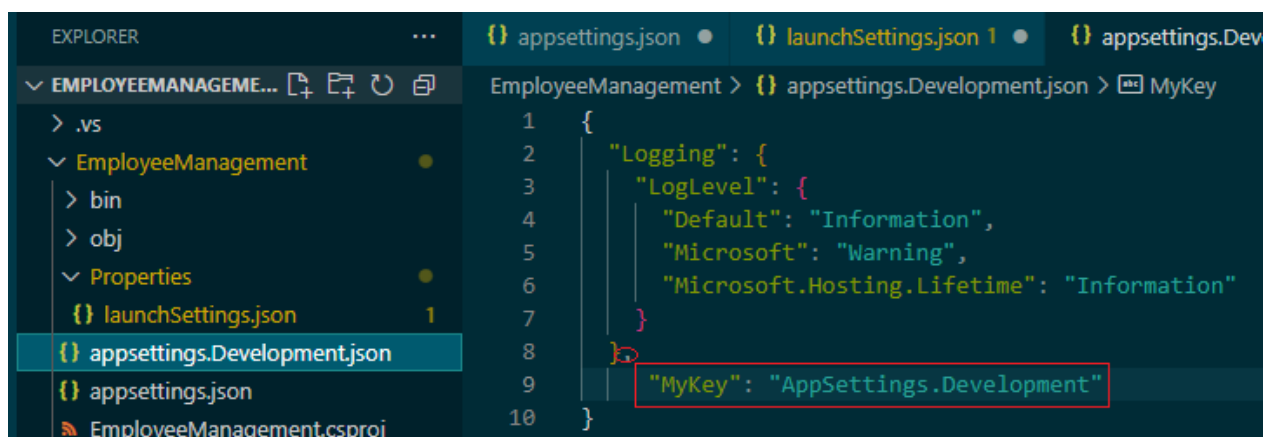


The screenshot shows the Visual Studio interface. In the Explorer, the 'Properties' folder of the 'EmployeeManagement' project is expanded, and 'launchSettings.json' is selected. The Code editor displays the content of 'launchSettings.json'. The 'profiles' section is expanded, showing the 'IIS Express' profile. The 'environmentVariables' object within this profile contains two entries: 'ASPNETCORE_ENVIRONMENT' set to 'Development' and 'MyKey' set to 'Launchsettings - IIS Express'. The 'MyKey' value is highlighted with a red box. Below the 'IIS Express' profile, the 'EmployeeManagement' section is also visible, with its 'environmentVariables' object containing 'ASPNETCORE_ENVIRONMENT' set to 'Development' and 'MyKey' set to 'LaunchSettings', also highlighted with a red box.

```
10  "profiles": {
11    "IIS Express": {
12      "commandName": "IISExpress",
13      "launchBrowser": true,
14      "environmentVariables": {
15        "ASPNETCORE_ENVIRONMENT": "Development",
16        "MyKey": "Launchsettings - IIS Express"
17      }
18    },
19    "EmployeeManagement": {
20      "commandName": "Project",
21      "dotnetRunMessages": true,
22      "launchBrowser": true,
23      "applicationUrl": "http://localhost:5000",
24      "environmentVariables": {
25        "ASPNETCORE_ENVIRONMENT": "Development",
26        "MyKey": "LaunchSettings"
27      }
28    }
29  }
```

Разбира се, в този файл, ще се изпълни този MyKey, който съответства на нашия адрес 5000.

"MyKey": "Hello from LaunchSettings.json!"



The screenshot shows the Visual Studio interface. In the Explorer, the 'Properties' folder of the 'EmployeeManagement' project is expanded, and 'appsettings.Development.json' is selected. The Code editor displays the content of 'appsettings.Development.json'. The 'Logging' section is expanded, showing the 'LogLevel' object. The 'environmentVariables' object within this profile contains two entries: 'ASPNETCORE_ENVIRONMENT' set to 'Development' and 'MyKey' set to 'AppSettings.Development', also highlighted with a red box.

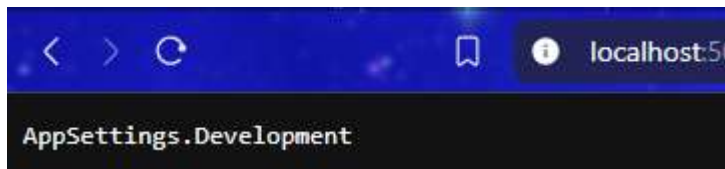
```
1  {
2    "Logging": {
3      "LogLevel": {
4        "Default": "Information",
5        "Microsoft": "Warning",
6        "Microsoft.Hosting.Lifetime": "Information"
7      }
8    },
9    "MyKey": "AppSettings.Development"
10 }
```

"MyKey": "Hello from AppSettings.Development!"

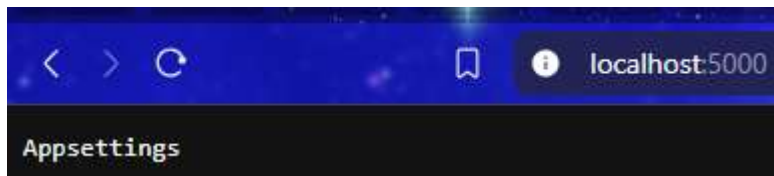
Когато и 3-те променливи са налични и изписани в тези 3 файла, с най-голям приоритет се вижда LaunchSettings.



Ако изтрием ключа от него и го оставим само в другите 2 файла:



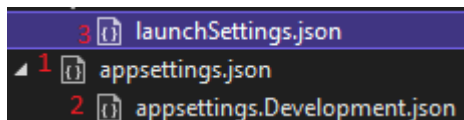
Ако изтрием ключа и от него, остава по подразбиране с най-малкия приоритет:



Не забравяме да изтриваме и запетайката на предходния ред след къдравата скоба.

Всеки път Strl + S за кода, Ctrl + C за терминала, dotnet run и Ctrl + R за браузъра.

Дотук разбрахме, че последователно се търси и изпълнява:



Връщаме се поотделно в 3-те файла и възстановяваме изтритите ключове на 2-та файла с Ctrl + Z (Undo) за действие назад, Ctrl + S за запазване и тестваме, след като в 3-те файла от Visual Code имаме записани 3 ключа, какво ще стане, ако си напишем ключ в терминала.

Не забравяме, че работещият терминал трябва да бъде само 1, затова, спираме процеса в Code с Ctrl + C.

```
Command Prompt - dotnet run dotnet run MyKey="CLI"
Microsoft Windows [Version 10.0.19045.3448]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Николета>cd /d D:\Exercises\12 class\EmployeeManagement\EmployeeManagement
D:\Exercises\12 class\EmployeeManagement\EmployeeManagement>dotnet run MyKey="CLI"
Using launch settings from D:\Exercises\12 class\EmployeeManagement\EmployeeManagement\Properties\launchSettings.json...
Building...
C:\Program Files\dotnet\sdk\9.0.200\Sdks\Microsoft.NET.Sdk\targets\Microsoft.NET.EolTargetFrameworks.targets(32,5): warn
ing NETSDK1138: The target framework 'net5.0' is out of support and will not receive security updates in the future. Ple
ase refer to https://aka.ms/dotnet-core-support for more information about the support policy.
C:\Program Files\dotnet\sdk\9.0.200\Sdks\Microsoft.NET.Sdk\targets\Microsoft.NET.EolTargetFrameworks.targets(32,5): warn
ing NETSDK1138: The target framework 'net5.0' is out of support and will not receive security updates in the future. Ple
ase refer to https://aka.ms/dotnet-core-support for more information about the support policy.
C:\Program Files\dotnet\sdk\9.0.200\Sdks\Microsoft.NET.Sdk\targets\Microsoft.NET.EolTargetFrameworks.targets(32,5): warn
ing NETSDK1138: The target framework 'net5.0' is out of support and will not receive security updates in the future. Ple
ase refer to https://aka.ms/dotnet-core-support for more information about the support policy.
info: Microsoft.Hosting.Lifetime[0]
      Now listening on: http://localhost:5000
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: D:\Exercises\12 class\EmployeeManagement\EmployeeManagement
```

cd /d

Пътят до Program.cs в нашата папка с проекта

```
dotnet run MyKey="Hello from CLI!"
```



С което доказваме, че терминалът е с най-най-най-висок приоритет!!!

<https://github.com/aspnet/MetaPackages/blob/release/2.2/src/Microsoft.AspNetCore.WebHost.cs> - с търсачката в страницата търсим `ConfigureAppConfiguration` – методът, в който е описана последователността при четене на конфигурационни източници. Ние можем да променим тази последователност, а също да добавим свой конфигурационен източник.